

LDNA Expressiveness (skeleton)

Definition 1 (Distinguishing power). A model class $\mathcal{M}$ *distinguishes* a pair $(\mathcal{G}_1, \mathcal{G}_2)$ if some parameters give them different outputs; $\mathcal{M}$ is *at least as expressive as* 1-WL if it distinguishes every pair 1-WL distinguishes, and *strictly more expressive* if it moreover distinguishes some pair 1-WL cannot.

1 Preliminaries

Recall the definition of a GNN and its permutation invariance and equivariance properties:

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ represent a graph, where $\mathcal{V}$ denotes the set of nodes, and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ denotes the set of edges. Let a graph-structured dataset be represented as $\mathcal{G} = (\mathbf{X}, \mathbf{A}, \mathbf{E})$, where

- $\mathbf{X} \in \mathbb{R}^{|\mathcal{V}| \times d}$ denotes the node feature matrix of the graph, with each row $\mathbf{x}_u \in \mathbb{R}^d$ being the d -dimensional features of node $u \in \mathcal{V}$;
- $\mathbf{A} \in \{0, 1\}^{|\mathcal{V}| \times |\mathcal{V}|}$ denotes the adjacency matrix of the graph, where $A_{u,v} = 1$ if $(u, v) \in \mathcal{E}$ and 0 otherwise;
- $\mathbf{E} \in \mathbb{R}^{|\mathcal{E}| \times d_e}$ denotes the optional edge feature matrix of the graph, with each row $\mathbf{e}_{u,v} \in \mathbb{R}^{d_e}$ being the d_e -dimensional features of edge $(u, v) \in \mathcal{E}$.

A typical Graph Neural Network (GNN) follows the Message Passing Neural Network (MP-GNN) [Gilmer et al., 2017] schema, which iteratively updates the d_h -dimensional latent node embedding $\mathbf{h}_u^{(l)} \in \mathbb{R}^{d_h}$ for every node $u \in \mathcal{V}$ from layer l to layer $l + 1$ via propagating messages through neighboring nodes $v \in \mathcal{N}_u$. This can be formalized as

$$\mathbf{h}_u^{(l+1)} = \text{UPDATE}\left(\mathbf{h}_u^{(l)}, \bigoplus_{v \in \mathcal{N}_u} \text{MESSAGE}\left(\mathbf{h}_u^{(l)}, \mathbf{h}_v^{(l)}, \mathbf{e}_{v,u}\right)\right) \quad (1)$$

where MESSAGE and UPDATE are learnable functions, such as Multi-Layer Perceptrons (MLPs), $\mathcal{N}_u = \{v \mid (v, u) \in \mathcal{E}\}$ is the (1-hop) neighborhood of node u , and $\bigoplus$ is a neighborhood aggregation function. After L iterations of aggregation, the representation $\mathbf{h}_u^{(L)}$ of node u encodes the structural information from its L -hop graph neighborhood. Finally, the graph embedding $\mathbf{h}_G \in \mathbb{R}^{d_G}$ can be obtained via a READOUT function:

$$\mathbf{h}_G = \text{READOUT}_{u \in \mathcal{V}}\left(\mathbf{h}_u^{(L)}\right) \quad (2)$$

which aggregates the node embeddings to obtain the entire graph’s representation $\mathbf{h}_G$.

2 LDNA Architecture

The main reason for limitations on the expressive power of GNNs with permutation-invariant aggregation is that they does not track node identities; meanwhile, different nodes with the same attributes will be always propagate identical messages, and are pooled to form the same aggregated messages. To address this limitation, we introduce a novel, learnable, and fully dynamic aggregation method, called *Learnable Dynamic Neighborhood Aggregation (DNA)*, that preserves permutation invariance while enhancing flexibility

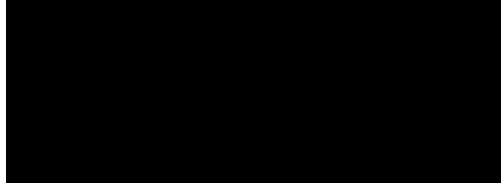


Figure 1: LDNA neighbourhoud-aggregation pipeline (stub).

and expressiveness. LDNA employs a parameterized transformation on the stacked neighbor messages as its aggregation mechanism, which can be implemented through a Deep Neural Network (DNN). As DNNs are not inherently permutation-invariant, in order for LDNA to maintain permutation invariance, it first sorts the incoming neighborhood messages in a canonical order. This ensures that regardless of the order in which the messages arrive, they are always placed in the same positions in the stacked messages. The specific metric for the canonical order does not affect LDNA’s performance. However, since the latent representations of the nodes evolve iteratively from layer to layer, it is preferable to choose an order based on constant values within the graph-structured data, such as the input node features. The message-aggregate-update pipeline for LDNA can therefore be formally defined as:

$$\mathbf{m}_{v,u}^{(l+1)} = \begin{cases} \phi_{\Theta}^{(l+1)}(\mathbf{h}_u^{(l)}, \mathbf{h}_v^{(l)}) & \text{(LDNA)} \\ \phi_{\Theta}^{(l+1)}(\mathbf{h}_u^{(l)}, \mathbf{h}_v^{(l)}, \mathbf{e}_{v,u}) & \text{(LDNA-e)} \end{cases} \quad (3)$$

$$\mathbf{h}_u^{(l+1)} = \gamma_{\Theta}^{(l+1)}\left(\mathbf{h}_u^{(l)}, \psi_{\Theta}^{(l+1)}\left(\left[\mathbf{m}_{\pi^*(v),u}^{(l+1)}\right]_{v \in \mathcal{N}_u}\right)\right) \quad (4)$$

where $\mathbf{m}_{u,v}$ denotes the message propagated through edge (u, v) , ϕ_{Θ} denotes the parameterized message function, ψ_{Θ} denotes the parameterized aggregation function, γ_{Θ} denotes the update function, and $\pi^*(\cdot)$ denotes the sorting function, a special permutation operation that maps node index v to its position in the sorted list according to a canonical order. We use Θ and θ to distinguish the practical characteristic that the message and update functions ϕ and γ may be modeled as DNNs, whereas the aggregation function typically only requires a shallow neural network.

Figure 1 illustrates the pipeline of LDNA’s neighborhood aggregation mechanism. This shows that apart from being use as part of the network defined in Equations (3) and (4), LDNA can also function as a standalone component, and seamlessly integrate to any aggregation-based GNN to enhance its expressive power, as defined by the following simplified formula:

$$\bigoplus_{v \in \mathcal{N}_u} \mathbf{m}_{v,u} = \psi_{\theta}\left(\left[\mathbf{m}_{\pi^*(v),u}\right]_{v \in \mathcal{N}_u}\right) \quad (5)$$

3 Proof of Theorem 1

Theorem 1. *LDNA is strictly more expressive than the 1-WL test, and hence any aggregation-based GNNs, if the parameters of the aggregation function are independently initialized under a continuous random distribution.*

Proof. Let $\{\{\cdot\}\}$ denote a multiset that allows for repetition of elements. Based on the definition of attributed regular graphs in Definition 5.8 in Li and Leskovec [2022], we describe the configuration of an attributed regular graph as a set of tuples:

$$\text{Config}(\mathcal{G}) = \{(C_i, C_j, r_{ij}) \mid 1 \leq i, j \leq k\} \quad (6)$$

where k is the number of categories of node attributes in $\mathcal{V}$, such that all nodes in $\mathcal{V}$ can be partitioned according to their attributes as $\mathcal{V} = \bigcup_{i=1}^k V_i$, with two nodes from the same category V_i sharing the same attributes, while nodes from different categories have distinct attributes; C_i represents the attribute of

nodes in category V_i ; and r_{ij} is the number of neighbors of a node in category V_i that belong to category V_j . Using Corollary 5.1 from Li and Leskovec [2022], we know that MP-GNN lacks the expressive power to distinguish two attributed regular graphs, $\mathcal{G}_1$ and $\mathcal{G}_2$, when they share the same configuration, i.e., $\text{Config}(\mathcal{G}_1) = \text{Config}(\mathcal{G}_2)$, and thus, predict their correct target outputs.

We restate the claim as follows: If LDNA considers two attributed regular graphs, $\mathcal{G}_1$ and $\mathcal{G}_2$, with configurations $\text{Config}(\mathcal{G}_1)$ and $\text{Config}(\mathcal{G}_2)$ as isomorphic, then 1-WL will also treat them as isomorphic. Following Definition 1, we can conclude that LDNA is at least as expressive than the 1-WL test. Intuitively, this means that LDNA partitions the vertices of each graph in a way that is at least as fine-grained as the partition produced by 1-WL.

We will prove this by induction, showing that there exists a model in the LDNA class that satisfies this property.

Base case: At $t = 0$, the statement holds because the initial vertex features are identical for both LDNA and 1-WL, i.e.,

$$\forall u \in \mathcal{V}_1 \cup \mathcal{V}_2. \mathbf{h}_u^{(0)} = c_u^{(0)} \quad (7)$$

Inductive step: Assume the statement holds for step $t - 1$, i.e.,

$$\forall u_1, v_2 \in \mathcal{V}_1 \cup \mathcal{V}_2. \mathbf{h}_{u_1}^{(t-1)} = \mathbf{h}_{u_2}^{(t-1)} \implies c_{u_1}^{(t-1)} = c_{u_2}^{(t-1)} \quad (8)$$

We now show that it also holds for step t .

The hidden representation of each vertex at step t is updated as follows:

$$\mathbf{h}_u^{(t)} = \text{UPDATE}^{(t)} \left(\mathbf{h}_u^{(t-1)}, \bigoplus_{v \in \mathcal{N}_u} \text{MESSAGE}^{(t)} \left(\mathbf{h}_u^{(t-1)}, \mathbf{h}_{\pi^*(v)}^{(t-1)}, \mathbf{e}_{\pi^*(v), u} \right) \right) \quad (9)$$

where $\text{UPDATE}^{(t)}$ is the update function. With a sufficient number of GNN layers, as shown in Equations (3) and (4), there exists a deep neural network function γ that acts as the update function, and this update function is injective of multisets. This injectivity is supported by the work in Cybenko [1989], which demonstrates that a deep neural network does not map distinct inputs to the same output in the feature space, ensuring a one-to-one mapping in the transformation. In addition, the composition of injective functions is injective.

Assume that $\mathbf{h}_{u_1}^{(t)} = \mathbf{h}_{u_2}^{(t)}$. Then:

1. From the induction hypothesis, we have $\mathbf{h}_{u_1}^{(t-1)} = \mathbf{h}_{u_2}^{(t-1)}$, which implies that $c_{u_1}^{(t-1)} = c_{u_2}^{(t-1)}$;
2. There exists a deep neural network $\psi \circ \phi$ which composes the message-aggregation function $\bigoplus^{(t)} \circ \text{MESSAGE}^{(t)}$ and is injective of multisets. Its injectivity is supported by the work in Cybenko [1989]. In addition, the composition of injective functions is injective. Thus, with respect to the multiset of hidden representations of the neighbors, we have

$$\{\{\mathbf{h}_{v_1}^{(t)} \mid v_1 \in \mathcal{N}_{u_1}\}\} = \{\{\mathbf{h}_{v_2}^{(t)} \mid v_2 \in \mathcal{N}_{u_2}\}\} \quad (10)$$

From the induction hypothesis, we know that $\mathbf{h}_{v_1}^{(t-1)} = \mathbf{h}_{v_2}^{(t-1)}$ implies $c_{v_1}^{(t-1)} = c_{v_2}^{(t-1)}$ for all $v_1 \in \mathcal{N}_{u_1}$ and $v_2 \in \mathcal{N}_{u_2}$. Therefore, we also have:

$$\{\{c_{v_1}^{(t-1)} \mid v_1 \in \mathcal{N}_{u_1}\}\} = \{\{c_{v_2}^{(t-1)} \mid v_2 \in \mathcal{N}_{u_2}\}\} \quad (11)$$

Thus, using the update rule of 1-WL, for each node $i \in \mathcal{V}$, we get:

$$c_u^{(t)} = \text{HASH} \left(\sum_{v \in \mathcal{N}_u \cup \{u\}} c_v^{(t-1)} \right) = \text{HASH} \left(c_u^{(t-1)}, \{\{c_v^{(t-1)} \mid v \in \mathcal{N}_u\}\} \right) \quad (12)$$

Since the multisets of the neighbors' features are the same for nodes u_1 and u_2 , we conclude that $c_{u_1}^{(t)} = c_{u_2}^{(t)}$. This completes the inductive step, proving that the statement holds for all $t \geq 0$. $\square$

4 Proof of Theorem 1

Throughout, node and edge feature alphabets are finite and graph sizes are bounded by N ; every statement asserts the existence of parameters realising a behaviour, in the sense of Definition 1. We write $\{\{\cdot\}\}$ for a multiset and $c_u^{(l)}$ for the 1-WL colour of node u after l rounds.

The stacked aggregation input. We first make the canonical order in Equation (4) precise. The paper’s prose—that the sorted messages “are always placed in the same positions in the stacked messages”—makes the argument of ψ_θ a fixed-size *slotted* object rather than a variable-length sorted list. Concretely, $[\mathbf{m}_{\pi^*(v),u}^{(l+1)}]_{v \in \mathcal{N}_u}$ is the matrix $\mathbf{S}_u \in \mathbb{R}^{N \times d_h}$ whose row $p \in \{0, \dots, N-1\}$ is

$$(\mathbf{S}_u)_p = \begin{cases} \mathbf{m}_{v,u}^{(l+1)} & \text{if } \pi^*(v) = p \text{ for some } v \in \mathcal{N}_u, \\ \mathbf{0} & \text{otherwise.} \end{cases}$$

Since the slot p occupied by a neighbour v is its canonical position $\pi^*(v)$, reading a slot recovers the identity of the neighbour placed there. This is exactly the information a permutation-invariant $\oplus$ discards, and the mechanism LDNA restores.

Theorem 2. *Let π^* be induced by a canonical labelling: an isomorphism-invariant bijection $\pi_G^* : \mathcal{V} \rightarrow \{0, \dots, n-1\}$ ($n = |\mathcal{V}|$) under which isomorphic graphs receive identical labelled graphs. Then LDNA (Equations (3) and (4)) is strictly more expressive than the 1-WL test, and hence than any MP-GNN with permutation-invariant aggregation [Xu et al., 2019, Morris et al., 2019].*

The same argument establishes the corresponding statement for LDNA-e against the edge-coloured 1-WL refinement, by reading $\mathbf{e}_{v,u}$ into every message.

Proof. We exhibit (Part 1) parameters making LDNA at least as discriminative as 1-WL and (Part 2) parameters separating a pair of graphs that 1-WL cannot; by Definition 1 the two together give strictness.

Part 1: at least as expressive as 1-WL. We choose parameters so that, for every layer l , the embedding $\mathbf{h}_u^{(l)}$ is an injective function of the 1-WL colour $c_u^{(l)}$, arguing by induction on l . At $l = 0$ both equal the input feature $\mathbf{x}_u$. Assume the claim at layer l . Because all quantities range over finite sets, the message map $\phi_\theta^{(l+1)}$ can encode the pair $(\mathbf{h}_v^{(l)}, \mathbf{e}_{v,u})$ injectively into $\mathbf{m}_{v,u}^{(l+1)}$. The aggregation $\psi_\theta^{(l+1)}$ is free to ignore the slot positions and sum the rows of $\mathbf{S}_u$; since a sum of suitable injective encodings is injective on multisets of bounded size (Xu et al., 2019, Lemma 5), $\psi_\theta^{(l+1)}(\mathbf{S}_u)$ can realise an injective function of $\{(\mathbf{h}_v^{(l)}, \mathbf{e}_{v,u}) \mid v \in \mathcal{N}_u\}$. The update $\gamma_\theta^{(l+1)}$ has an explicit self-argument $\mathbf{h}_u^{(l)}$, so it can injectively combine the node’s own colour with this aggregate; an empty neighbourhood is harmless, as the self-argument then determines $\mathbf{h}_u^{(l+1)}$ on its own. The result is an injective function of $(c_u^{(l)}, \{c_v^{(l)} \mid v \in \mathcal{N}_u\})$, which is the argument of the injective 1-WL update; hence $\mathbf{h}_u^{(l+1)}$ is an injective function of $c_u^{(l+1)}$. Finally, a READOUT summing injective encodings of the final embeddings is injective on the colour histogram [Xu et al., 2019], so whenever 1-WL assigns two graphs different colour histograms LDNA assigns them different graph embeddings.

Part 2: strictly more expressive than 1-WL. Let $\mathcal{G}_1 = C_6$ be the 6-cycle and $\mathcal{G}_2 = C_3 \sqcup C_3$ two disjoint triangles, both with a single constant node feature and no edge features.

(i) *1-WL cannot separate them.* Both are 2-regular with identical initial colours, so by induction all nodes keep one common colour: if every node has colour a at round l , each has neighbour multiset $\{\{a, a\}\}$ and the 1-WL hash returns a single colour at round $l+1$. The two graphs thus have equal colour histograms at every round and are 1-WL-indistinguishable.

(ii) *A canonical labelling.* A canonizer fixes one representative labelling per isomorphism class; take the cyclic order $0, 1, \dots, 5$ along C_6 and the consecutive triangles $\{0, 1, 2\}, \{3, 4, 5\}$ for $C_3 \sqcup C_3$, so $\pi^*(v) = v$.

(iii) *Parameters that separate them.* On a single channel set $\phi_\theta \equiv 1$, so every message equals 1 and $(\mathbf{S}_u)_p = 1$ precisely when the neighbour at canonical position p lies in $\mathcal{N}_u$. Let ψ_θ be the shallow per-slot

linear map with weight 2^p on slot p — slots being indexed by the *global* canonical position, per the stacked form above, not by the rank within the neighbourhood; then

$$\psi_{\theta}(\mathbf{S}_u) = \sum_{v \in \mathcal{N}_u} 2^{\pi^*(v)}.$$

Reading off the neighbourhoods gives the node-level multisets

$$C_6 : \{\{34, 5, 10, 20, 40, 17\}\}, \quad C_3 \sqcup C_3 : \{\{6, 5, 3, 48, 40, 24\}\};$$

for instance node 0 of C_6 has neighbours at slots 1 and 5, giving $2^1 + 2^5 = 34$, whereas node 0 of $C_3 \sqcup C_3$ has neighbours at slots 1 and 2, giving $2^1 + 2^2 = 6$. The two multisets differ (34 occurs only in the first), even though their element-sums coincide (126 each), so a plain summation aggregator would not suffice and the slot-weighting is what separates them.

(iv) *Conclusion.* Taking γ_{θ} and the READOUT injective as in Part 1, the graph embedding is an injective function of the node-level multiset above; as the two multisets differ, LDNA assigns $\mathcal{G}_1$ and $\mathcal{G}_2$ different embeddings. Since 1-WL cannot separate them, LDNA is strictly more expressive than 1-WL, and hence than any MP-GNN with permutation-invariant aggregation. $\square$

Remark 1 (The canonical labelling is essential). The extra power comes from π^* individualizing nodes, not from the stacked form alone. If the aggregation input were reduced to the sorted sequence of message *contents* (ties left unbroken), that sequence is a canonical encoding of the multiset $\{\{\mathbf{m}_{v,u}\}\}$, so $\psi \circ \text{sort}$ is a multiset function and the model collapses to exactly 1-WL. The same collapse occurs for any order computed from 1-WL-invariant node quantities, which fold into the colours; and on vertex-transitive 1-WL-hard pairs—the 4×4 rook’s graph versus the Shrikhande graph—every invariant order ties all nodes, leaving $\oplus$ symmetric. An individualizing canonical labelling, obtainable by completing the feature sort through individualization–refinement [McKay and Piperno, 2014] (with quasi-polynomial worst-case cost [Babai, 2016]), is what supplies the node identities that break these ties.

Remark 2 (Random initialization). Injectivity on a finite domain is generic: for each map involved, the parameter settings that make two given distinct inputs collide form a measure-zero set, and there are finitely many input pairs. With parameters drawn independently from a continuous distribution, every layer map is therefore almost surely injective, and then each embedding $\mathbf{h}_u^{(l)}$ *refines* the colour $c_u^{(l)}$: the embedding determines the colour, though a position-aware random ψ_{θ} may split colour classes further. Refined histograms separate whatever the colour histograms separate, so a randomly initialized LDNA attains at least 1-WL discriminative power almost surely, without any training. The strictness in Part 2 instead relies on explicit deterministic parameters and needs no randomness; this complements random node initialization [Abboud et al., 2021], which randomizes the inputs rather than the aggregator.

Remark 3 (Permutation invariance). Although π^* orders the nodes, the model output is a deterministic graph invariant: the canonical labelling depends only on the isomorphism class, and any choice within an automorphism orbit yields the same labelled graph—hence the same stacked inputs $\mathbf{S}_u$ and the same output. The sort that secures permutation invariance and the labelling that supplies node identities are therefore one and the same object.

References

- Ralph Abboud, İsmail İlkan Ceylan, Martin Grohe, and Thomas Lukasiewicz. The surprising power of graph neural networks with random node initialization. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 2112–2118, 2021.
- László Babai. Graph isomorphism in quasipolynomial time. In *Proceedings of the 48th Annual ACM Symposium on Theory of Computing (STOC)*, pages 684–697, 2016.
- George Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.

- Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 1263–1272, 2017.
- Pan Li and Jure Leskovec. The expressive power of graph neural networks. In Lingfei Wu, Peng Cui, Jian Pei, and Liang Zhao, editors, *Graph Neural Networks: Foundations, Frontiers, and Applications*, pages 63–98. Springer, 2022.
- Brendan D. McKay and Adolfo Piperno. Practical graph isomorphism, II. *Journal of Symbolic Computation*, 60:94–112, 2014.
- Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and Leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 4602–4609, 2019.
- Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations (ICLR)*, 2019.